```
        new City{ Id = 2, Name = "York" , Country = new Country{
                  ➥ CountryName = "United Kingdom"} },
        new City{ Id = 3, Name = "Venice", Country = new Country{
                  ➥ CountryName = "Italy"} },
        new City{ Id = 4, Name = "Rome", Country = new Country{
                  ➥ CountryName = "Italy" } },
        new City{ Id = 5, Name = "Madrid" , Country = new Country{
                  ➥ CountryName = "Spain" } },
        new City{ Id = 5, Name = "Barcelona" , Country = new Country{
                  ➥ CountryName = "Spain" } },
        new City{ Id = 5, Name = "Cadiz" , Country = new Country{
                  ➥ CountryName = "Spain" } }
    };
    return new SelectList(cities, nameof(City.Id), nameof(City.Name),
                          ➥ null, "Country.CountryName");                    ❶
}
```

❶ You use a literal string to represent the DataGroupField when working with complex objects.

This time you cannot use the `nameof` operator to represent the `DataGroupField` name. You must pass in the full property name as a string; otherwise, it will not be found, resulting in a `NullReferenceException` at run time.

6.3.4 Binding enumerations

In most real-life applications, the options you populate your select lists with will come from a database, especially if they are likely to change. When the range of options is naturally limited, you might decide that an enumeration can handle them instead. For example, there are a limited number of ways to reasonably describe their work status: employed full-time, employed part-time, self-employed, seeking work, retired, or in education. So you might create an enumeration named `WorkStatus` that has members representing all possible options.

In your application, the rating that is applied to a property that one can stay in is a good candidate for an enumeration. Ratings range from one star to